Indexer and Query Processor

Programming Assignment #2, Information Retrieval

Vinicius Trindade - 2020007112
Universidade Federal de Minas Gerais
Belo Horizonte, MG, Brazil
viniciustda@ufmg.br

Abstract

We implement an external indexer and a conjunctive query processor in Python for a corpus of 4,641,784 Wikipedia named entities. The indexer follows the SPIMI workflow: a thread pool pre-processes documents into an in-memory dictionary that is flushed once its postings reach a budget-derived threshold, and a k-way merge produces a single inverted index with γ/unary-encoded postings. The query processor does document-at-a-time conjunctive matching under two rankers, TF-IDF (pivoted-length normalization) and BM25. The full corpus was indexed in 14 min 44 s within a strict 1 GB budget (peak RSS 0.67 GB), producing 410 MB of index over 2.27 M inverted lists. The five test queries return in 0.9–4.5 s each.

Keywords

Inverted index, SPIMI, external merge sort, γ-coding, document-at-a-time, BM25, TF-IDF, Porter stemming

1 Introduction

The implementation runs exactly as the assignment specifies: `python3 indexer.py -m <MEMORY> -c <CORPUS> -i <INDEX>` for indexing and `python3 processor.py -i <INDEX> -q <QUERIES> -r <TFIDF|BM25>` for retrieval, over the 4.6 M-entity Wikipedia JSONL corpus from Kaggle.

The code is a thin CLI shim (`indexer.py`, `processor.py`) over a package `tp2/` that hosts the logic: shared modules `text.py` (pre-processing), `codec.py` (γ/unary bit-packing), `config.py` and `memory.py`, plus the sub-packages `tp2/indexer` and `tp2/processor`, each with one orchestration runner.

Section 2 describes data structures, algorithms, and complexity for indexing and querying. Section 3 reports empirical efficiency. Section 4 characterizes the produced index and the retrieved results. Section 5 discloses the use of AI tools.

2 Data Structures and Algorithms

2.1 Pre-processing

A single function `analyze()` in `text.py` is shared by the indexer and the processor, ensuring queries are pre-processed the same way as documents. Each input string undergoes (i) Unicode NFKD normalization with ASCII strip, (ii) lowercasing, (iii) tokenization by the regex `[a-z0-9]{3,}`, accepting any sequence of three or more alphanumeric characters, (iv) English stopword removal (NLTK list, applied *before* stemming so that stems of stopwords such as `wa` or `ha` do not pollute the lexicon), and (v) Porter stemming, memoized with a 200,000 entry LRU cache to exploit the Zipfian distribution of input tokens. The pipeline is $O(n)$ in the document size. The stemmer cache reaches a hit-rate above 70%, eliminating most repeated work.

2.2 Indexer

The indexer uses the SPIMI pattern. The in-memory accumulator `MemoryIndexBuilder` maintains a `dict[str, list[Posting]]`, supporting $O(1)$ amortized inserts. Documents are streamed from the JSONL corpus (line-by-line, tolerant to malformed lines), batched into groups of 2,048, and dispatched to worker threads that pre-process them and return one (doc_len, term_tf_map) tuple each.

The accumulator is flushed to a partial run once its posting count reaches a budget-derived threshold, the number of postings whose estimated footprint fills half the budget. We trigger on the *posting count*, not a live RSS reading: the glibc allocator retains freed heap pages after a flush, so an RSS-based trigger would stay pinned at its high-water mark and fire on every batch. The posting count is deterministic and portable. RSS is still monitored as a 90%-of-budget safety ceiling. Each run is sorted lexicographically before being written. Since the orchestrator assigns docids monotonically across batches, consecutive runs cover disjoint docid intervals, which enables the simple concatenation merge in the next phase.

The external k-way merge keeps a min-heap of (term, run_idx), drains the smallest term from *all* runs holding it, and appends the union of its postings (already in docid order) to `inverted_index.bin`. A lexicon entry (offset, length, df, ctf) goes to `lexicon.bin` in parallel. The merge is $O(P \log K)$ for P postings and K runs.

Compression uses γ-codes for the first docid and docid gaps, and unary for term frequencies, with a γ fallback when tf > 32 to avoid long unary codes. The codec is implemented manually in `codec.py`. Decoding is symmetric, exposed as a streaming `PostingIterator` used by DAAT.

The document index `doc_index.bin` is written in streaming mode during phase 1 as one fixed 24-byte record per document (external ID, length, and a reserved field), allowing $O(1)$ lookup of $|d|$ by docid via a memory-mapped slice in the processor.

2.3 Memory budget

`MemoryGuard` in `memory.py` reads the process RSS through `psutil`. Because the indexer parallelizes with threads (Section 2.5) there are no child processes, so this single reading covers the whole implementation. RSS is a safety ceiling at 90% of the budget, while the primary flush trigger remains the posting count above, which keeps the design portable, with no platform-specific calls such as `malloc_trim`.

2.4 Query Processor

Each query line is pre-processed by the same `analyze()` function. Lookup of every resulting stem in the in-memory lexicon yields the list of `LexEntry` records. A single missing stem makes the conjunction unsatisfiable and the query returns an empty result, per the spec's conjunctive matching policy.

Otherwise the candidate lists are opened as `PostingIterators` over the memory-mapped index and sorted by *df*, so the rarest term leads. The DAAT loop takes the leader's docid as the pivot, advances every other iterator until its docid is ≥ pivot, scores the document only when all iterators agree, and pushes (score, docid) into a top-k min-heap ($k = 10$). Total cost is $O(\sum_i \mathrm{df}_i)$ posting reads, dominated by the rarest list.

Two rankers are provided: TF-IDF with pivoted length normalization and Okapi BM25 ($k_1 = 1.2$). Both damp the term frequency by the same document-length factor ($b = 0.75$) and weight it by the smoothed inverse document frequency $\log((N+1)/df_w)$.

2.5 Parallelization

The indexer parallelizes its pre-processing stage with a `ThreadPool Executor` (default `cpu_count() − 1` threads, overridable via `TP2_INDEXER_WORKERS`). A sliding window of twice the thread count bounds the batches in flight. Results are consumed in submission order to preserve the monotonic docid assignment.

CPython's GIL serializes bytecode execution, and our pre-processing is bytecode-bound (regex, stopword filtering, Porter stemming), so threads give only a modest speedup (Section 3). We still chose threads over `multiprocessing` deliberately: processes bypass the GIL but their pickle traffic fragments the heap and pushes RSS far over the 1 GB budget (Section 3). The thread version parallelizes by design *and* stays both within budget and portable.

The query processor likewise uses a `ThreadPoolExecutor`, where threads are additionally a performance win: per-query work is light, the memory-mapped index is shared between threads at no copy cost, and the GIL is released during `mmap` page faults. Output order matches submission order, keeping results reproducible.

3 Empirical Efficiency

The experiments below ran on a laptop with an Intel Core i7-10750H (6 physical / 12 logical cores, 16 GB RAM) running Ubuntu 22.04 with Python 3.14, NLTK 3.9.4 and psutil 7.2.0, running `python3 indexer.py -m 1024 -c corpus.jsonl -i index` as specified.

3.1 Step timings

For the main run (11 threads), Phase 1 (read + pre-process + accumulate) takes 576 s and dominates: it tokenizes, stopword-filters and Porter-stems every one of the 4.64 M documents. SPIMI flushes add 86 s over the 27 emitted runs. Phase 2, the k-way merge with γ/unary encoding, costs 308 s, and finalization 0.09 s. Total wall-clock time was 884 s (14 min 44 s).

3.2 Speedup and the threads-vs-processes trade-off

Figure 1 plots indexing speedup vs. worker count. With the submitted thread-based configuration, speedup peaks at a modest 1.05× at two threads, then flattens: the GIL serializes the CPU-bound pre-processing, so extra threads add only scheduling overhead. The small two-thread gain comes from I/O (corpus reads, run-file writes) overlapping with computation while the GIL is released.

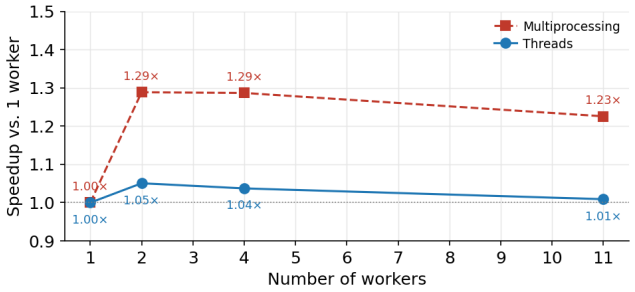


Figure 1: Indexing speedup vs. worker count. Threads (solid, submitted) stay within the 1 GB budget but the GIL caps speedup near 1.0×. Multiprocessing (dashed) gives a real 1.29× but its peak RSS (3.1–6.7 GB) violates the budget.

The dashed line is an equivalent `multiprocessing` implementation: processes bypass the GIL and reach a real 1.29×, but the inter-process pickle traffic fragments the heap so severely that peak RSS reaches 3.1 GB with two processes and 6.7 GB with eleven, 3 to 6× over the 1 GB budget. Recovering it would need `libc.malloc_trim`, a non-portable glibc call the assignment advises against. We therefore ship the thread-based version and report multiprocessing only as a trade-off analysis.

3.3 Memory compliance

Peak RSS over the entire run stayed between 519 MB (1 thread) and 673 MB (11 threads, which hold more batches in flight), always well under the 1024 MB budget. The posting-count flush trigger emitted exactly 27 partial runs in every configuration, confirming it is deterministic and independent of allocator behaviour.

3.4 Query throughput

Per-query latency (Table 2) ranges from 0.9 s to 4.5 s. This time is almost entirely the DAAT posting traversal: query pre-processing (`analyze()`) and the in-memory lexicon lookups are O(query length) and cost well under a millisecond. Because DAAT is dominated by the leader's (rarest) posting list, latency tracks $\min_w df_w$ rather than query length: short queries on rare terms (*nolan movies*) are faster than queries containing very common stems (*19th century female authors*, where `centuri` has df > 350k).

4 Index and Query Characterization

4.1 Index characterization

Table 1 summarizes the produced index. The 132 M postings encoded in 230 MB give roughly 14.6 bits per posting on average, consistent with the expected gain of γ on small gaps and unary on small term frequencies. The total index is just under 20% of the size of the raw 2.1 GB JSONL.

Table 1: Corpus and index statistics.

Documents indexed (N)	4,641,784
Unique terms (inverted lists)	2,266,400
Total postings	132,401,251
Total tokens after pre-processing	194,996,142
Average document length ($\overline{\lvert d \rvert}$, tokens)	42.01
Average list size (postings/term)	58.42
Number of partial runs emitted	27
`inverted_index.bin`	230 MB
`lexicon.bin`	74 MB
`doc_index.bin`	106 MB
Total index size	**410 MB**
Index size as % of raw JSONL (2.1 GB)	19.5%
Bits per posting (γ/unary)	$\approx$ 14.6

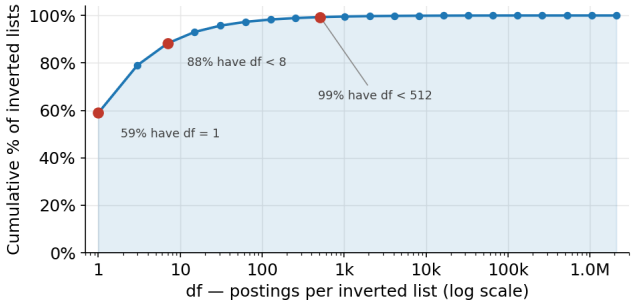


Figure 2: Cumulative distribution of document frequency over the 2.27 M inverted lists. A point (d, y) reads as "y% of the inverted lists have df $\leq d$". The x-axis is logarithmic in df, the y-axis a linear percentage. The steep rise and early saturation reflect a strongly skewed, Zipf-like distribution.

Table 2: Per-query characterization on the five test queries. *Stems* counts query terms that survive stopword removal and the 3-character filter. *Matched* is the number of documents satisfying the conjunctive query. The processor ranks these and outputs the top $k = 10$. *Range* is the score min–max over the returned documents (omitted when none matched). *Time* is wall-clock latency in milliseconds.

			TFIDF		BM25	
Query	Stems	Matched	Range	Time	Range	Time
nolan movies	3	2	8.1–10.4	1115	17.0–19.7	876
radiohead albums	2	145	10.5–13.8	2449	20.4–22.3	2762
physics nobel…	5	0	–	4014	–	4385
german univ.	2	11,669	7.4–8.9	4078	10.6–10.9	4489
19th c. authors	4	47	7.6–9.8	4169	16.6–19.1	3285

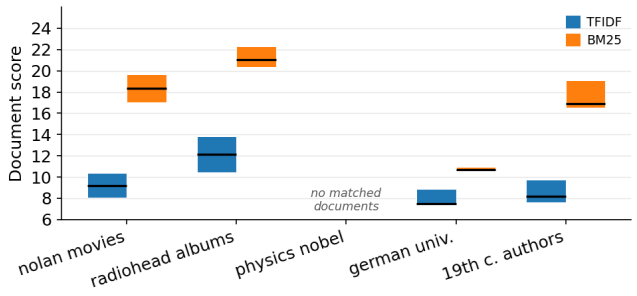


Figure 3: Score min–max range (colored bars) and median (black line) per query under TF-IDF (blue) and BM25 (orange). The query with no matched documents carries no bars. BM25 produces consistently higher absolute scores, but the two rankers agree on the top documents for most queries.

The document-frequency distribution (Fig. 2) is strongly skewed toward rare terms, the Zipfian behaviour expected of natural-language text. The cumulative curve shows that 1,335,186 terms, 59% of the lexicon, are hapax legomena (df = 1), with 88% of the inverted lists having df < 8 and 99% having df < 512, so only about 1% of the terms reach the high-df regime. The most frequent stem is `birth` with df = 1,147,137, followed by `peopl`, `born`, `live`, `state`. This is a direct reflection of the corpus, which is dominated by biographical named entities.

4.2 Query results

Table 2 summarizes the result set for each of the five test queries under both rankers. Two observations are worth highlighting:

First, *physics nobel winners since 2000* returns zero matches: it becomes the five-stem conjunction `physic` ∧ `nobel` ∧ `winner` ∧ `sinc` ∧ `2000` (`since` is not an NLTK stopword), and the strict DAAT-conjunctive policy demands all five in one document. In practice `sinc` and the literal `2000` rarely co-occur with the rest in Wikipedia entity descriptions. An OR-style retrieval, which the spec forbids, would return many documents. Second, *german cs universities* reduces to `german` ∧ `univers` because `cs` is dropped by the 3-character minimum. This widens the result set beyond strict computer-science institutions (e.g. the German-Jordanian University).

Figure 3 compares the score ranges. BM25 produces higher absolute scores than TF-IDF for every query, but the two formulas sit on different scales, so absolute values are not comparable across rankers. For *nolan movies* and *19th century female authors*, TF-IDF and BM25 return the same first two documents. They disagree for *radiohead albums* and *german cs universities*, ranking different documents and even choosing a different best match.

The produced index is available on Google Drive.

5 Use of AI Tools

Claude AI was used as a development assistant for (i) plan the implementation order; (ii) review the implementation to ensure that all specification requirements were met; (iii) debug; (iv) understand some results obtained after running the code; and (v) translation of some phrases for this documentation. All code was written and reviewed by the student, with AI suggestions validated against the course material.